[인공지능]

Ensemble Learning

Eunsong Kang
eskang@kangwon.ac.kr

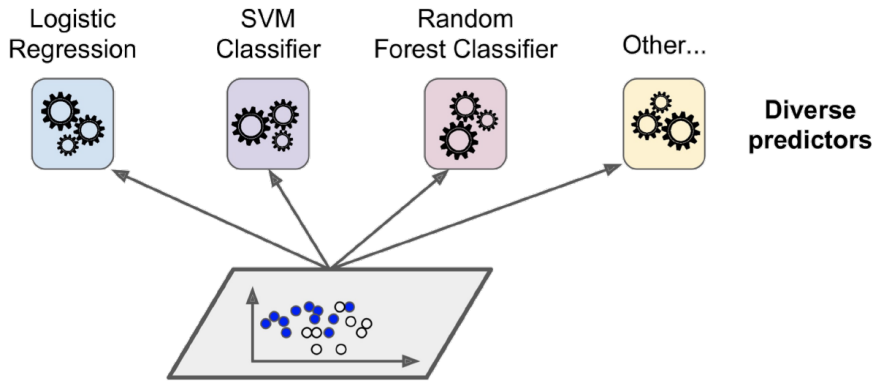
2026. 10. 22.

Ensemble Learning

- Ensemble learning is a method of combining multiple individual models to create an optimal model with improved generalization.
- Most popular ensemble methods
  - Voting
  - Bagging (*i.e.*, random forests)
  - Boosting (*i.e.*, Adaboost, XGBoost)
  - Stacking

Voting Classifier

- Suppose you have trained a few classifiers, each one achieving about 80% accuracy.



Training diverse classifiers

Hard Voting Classifier

- A simple way to create an even better classifier is to aggregate the predictions of each classifier and predict the class that gets the most votes.
  - Often achieves a higher accuracy than the best classifier in the ensemble.

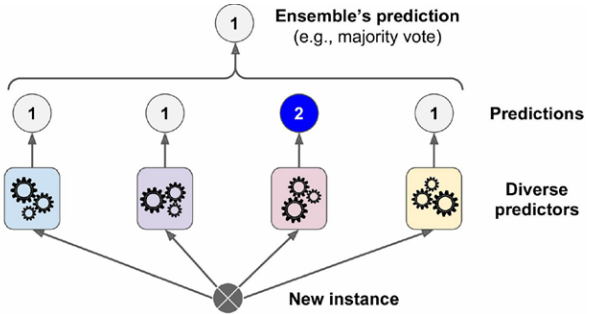


Figure 7-2. Hard voting classifier predictions

Hard voting classifier predictions

- Even if each classifier is a weak learner, the ensemble can still be a strong learning, provided there is a sufficient number of weak learners and they are sufficiently diverse.

Suppose that we have 5 independent classifiers for majority voting.



If the accuracy is 70% for each classifier, the accuracy of majority voting is:

$$\binom{5}{5}.7^5 + \binom{5}{4}.7^4(.3) + \binom{5}{3}.7^3(.3^2) = 0.8369$$

If there are 101 such classifiers, the majority voting accuracy is 0.99.

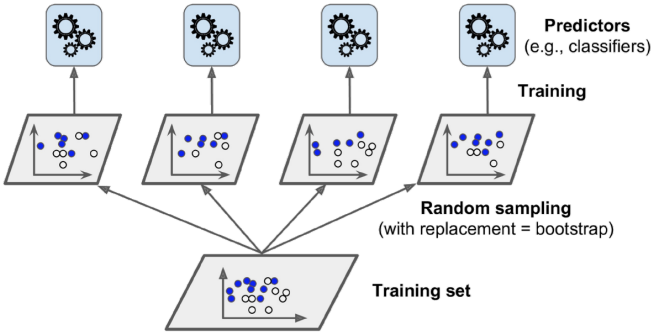
Only true if all classifiers are perfectly independent, making uncorrelated errors, which is clearly not the case since they are trained on the same data

Soft Voting Classifier

- Soft Voting is a method in which the predicted class probabilities from each classifier are averaged, and the class with the highest average probability is chosen as the final prediction.
- In other words, it is a more refined approach than hard voting because it utilizes the probability outputs of each classifier.

Bagging and Pasting

- Another approach is to use the same training algorithm for every predictor, but to train them on different random subsets of the training set.
  - Bagging (Bootstrapping): sampling with replacement
  - Pasting: sampling without replacement



Pasting/bagging training set sampling and training

Why Bagging Lowers Variance?

- Generally, the net result is that the ensemble has a similar bias but a lower variance than a single predictor trained on the original training set.
- Assume that we measure a random variable $x \sim \mathcal{N}(\mu, \sigma^2)$. If we only have a single measurement x_1,

$$E[x_1] = \mu, \quad \text{and} \quad \mathrm{Var}(x_1) = \sigma^2$$

If we have measured x k times, and the value of x is estimated as

$$\frac{x_1 + x_2 + \cdots + x_k}{K},$$

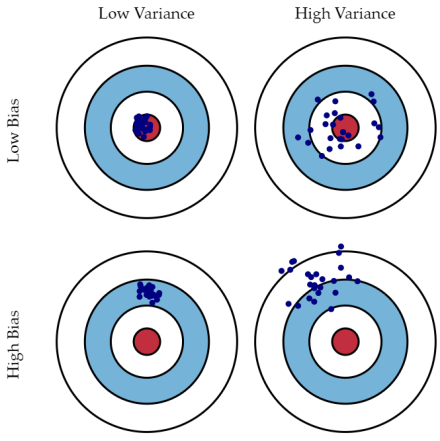
then

$$E\left[\frac{x_1 + x_2 + \cdots + x_k}{K}\right] = \mu;$$

$$\mathrm{Var}\left(\frac{x_1 + x_2 + \cdots + x_k}{K}\right) = \frac{K\sigma^2}{K^2} = \frac{\sigma^2}{K}$$

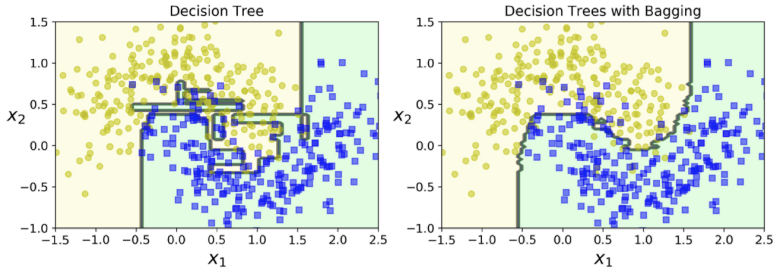
- Thus, bagging results in a model with smaller variance.

[Review] Bias and Variance



http://scott.fortmann-roe.com

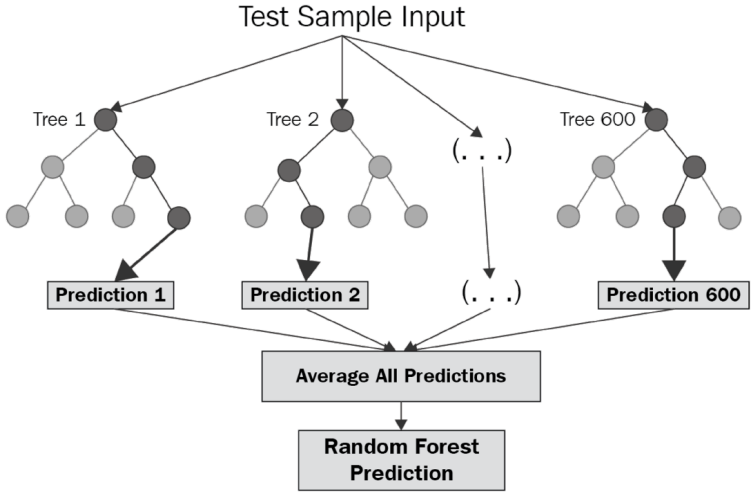
KNU
KANGWON NATIONAL UNIVERSITY



A single decision tree versus a bagging ensemble of 500 trees

Random Forest

- A **random forest** is an ensemble of decision trees, generally trained via the bagging method (or sometimes pasting).
- The random forest algorithm introduces extra randomness when growing trees; instead of searching for the very best feature when splitting a node, it searches for the best feature among a random subset of features.
  - This results in a greater tree diversity, which (once again) trades a higher bias for a lower variance, generally yielding an overall better model.
- Another great quality of random forests is that they make it easy to measure the relative importance of each feature.
  - Measures a feature's importance by looking at how much the tree nodes that use that feature reduce impurity on average (across all trees in the forest).


Test Sample Input
Tree 1
Tree 2
(. . .)
Tree 600
Prediction 1
Prediction 2
(. . .)
Prediction 600
Average All Predictions
Random Forest Prediction

Random Forest Variable Importance

1. Compute the out of bag (OOB) error for the original dataset (e_i).
2. Compute the OOB error for the dataset in which the variable x_i is permuted (p_i).
3. Compute the variable importance based on the mean and standard deviation of ($p_i - e_i$) over all trees in the population.

* Out-of-Bag (OOB) Samples: Training instances not selected during bootstrap sampling.

- A feature is important if randomizing it on OOB data consistently increases the prediction error.
- For tree t and feature i:

$$e^{(t)} = \text{OOBError}^{(t)} \text{ (original)}, \qquad p_i^{(t)} = \text{OOBError}^{(t)} \text{ (after permuting feature } i\text{)},$$

$$d_i^{(t)} \;=\; p_i^{(t)} - e^{(t)}.$$

- With T trees,

$$\bar{d}_i = \frac{1}{T}\sum_{t=1}^{T} d_i^{(t)}, \qquad s_i^2 = \frac{1}{T-1}\sum_{t=1}^{T}\left(d_i^{(t)} - \bar{d}_i\right)^2.$$

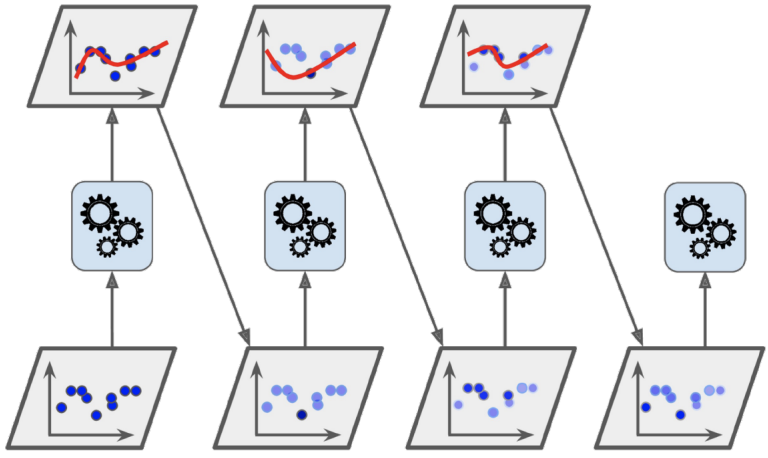
Importance score: $\qquad v_i = \dfrac{\bar{d}_i}{s_i}.$

Boosting

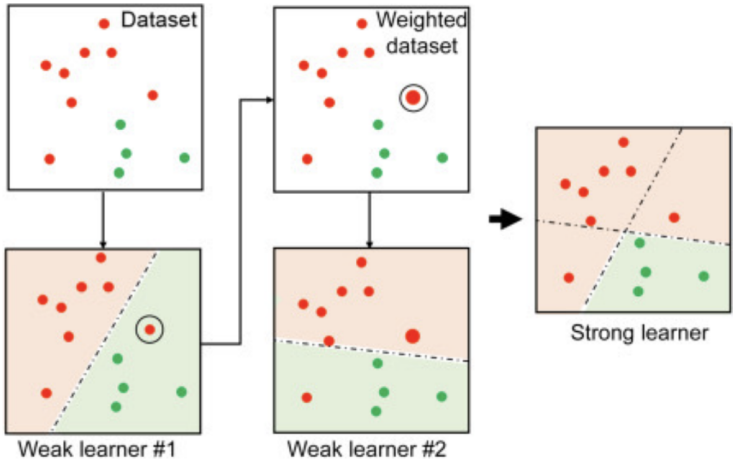
- **Boosting** refers to any ensemble methods that can combine several weak learners into a strong learner.
- The general idea of most boosting methods is to train predictors sequentially, each trying to correct its predecessor.
- *AdaBoost*, *Gradient Boosting*, *XGBoost*, *etc.*.

AdaBoost

- Pay more attention to training instances that were underfitted.
- New predictors increasingly focus on hard cases.
- Training process:
  1. Train a base classifier (e.g., Decision tree) on the training set.
  2. Increase the weights of misclassified instances.
  3. Train the next classifier on the updated weights.
  4. Repeat and combine all classifiers into an ensemble.



AdaBoost sequential training with instance weight updates



https://injo.tistory.com/31

AdaBoost: Weighted Error & Predictor Weight

- Each instance has an initial weight $w^{(i)} = \frac{1}{m}$, where m is the number of training instances.
- A predictor is trained and its **weighted error rate** is calculated:

$$r_j = \frac{\sum_{i=1}^{m} w^{(i)} \mathbf{1}(\hat{y}_j^{(i)} \neq y^{(i)})}{\sum_{i=1}^{m} w^{(i)}}$$

  where $\hat{y}_j^{(i)}$ is the prediction of the j-th predictor for the i-th instance.

- The predictor's weight α_j is then computed using:

$$\alpha_j = \eta \log\left(\frac{1 - r_j}{r_j}\right)$$

  - η: learning rate
  - If the predictor is accurate, α_j is high.
  - If it's worse than random guessing, $\alpha_j < 0$.

AdaBoost: Weight Update & Final Prediction

- Instance weights are updated:

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \cdot \exp(\alpha_j) & \text{if } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases}$$

  - Normalize all weights so that $\sum_{i=1}^{m} w^{(i)} = 1$

- Train a new predictor with updated weights.
- Repeat until desired number of predictors is reached or perfect predictor is found.
- Final Prediction: Weighted majority vote

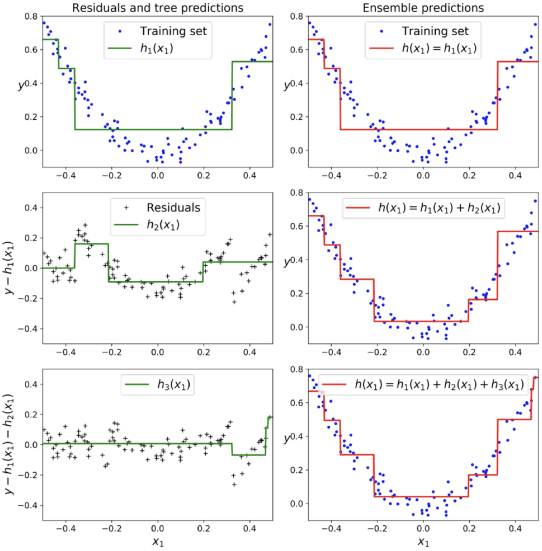
$$\hat{y}(\mathbf{x}) = \arg\max_{k} \sum_{j=1, \hat{y}_j(\mathbf{x})=k}^{N} \alpha_j$$

where N is the total number of predictors and k is the index of class.

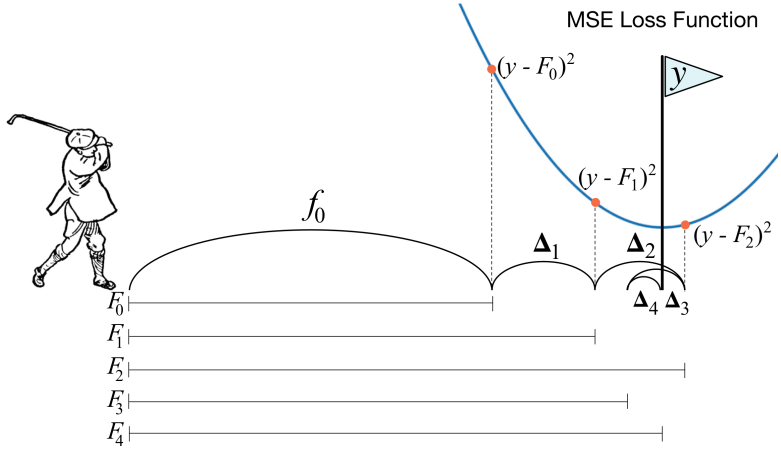
KNU KANGWON NATIONAL UNIVERSITY

Gradient Boosting

- **Gradient boosting** is a popular boosting algorithm.
  - Like AdaBoost, it builds an ensemble by sequentially adding predictors.
  - Each new predictor corrects the errors of its predecessor.
- Instead of adjusting instance weights, gradient boosting fits the new predictor to the **residual errors** made by the previous one.
- Gradient boosting can be used for different problems: regression, classification, and ranking.



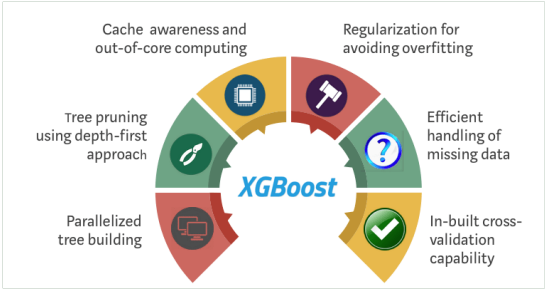
Gradient Boosting



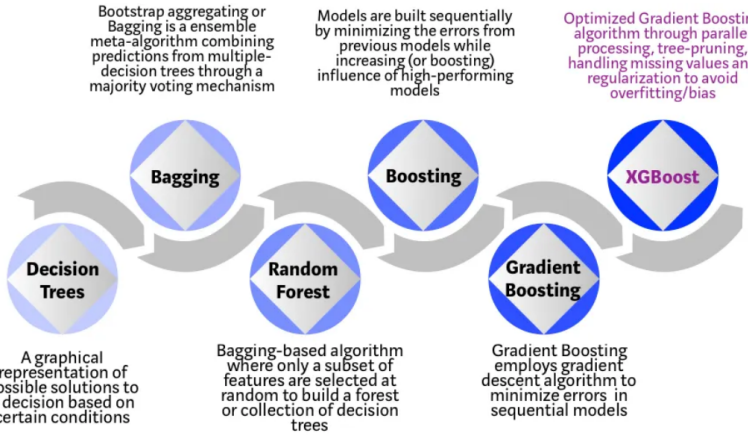
Source from Github

XGBoost

- **XGBoost** (Extreme Gradient Boosting) is a gradient boosting algorithm that builds trees sequentially to correct previous errors, using advanced optimization and regularization to reduce overfitting and enhance performance.



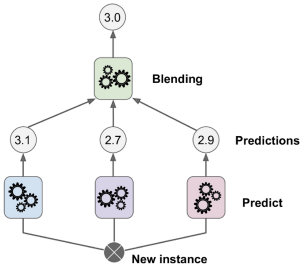
Advantages of XGBoost



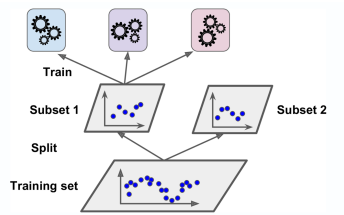
https://medium.com/data-science/https-medium-com-vishalmorde-xgboost-algorithm-long-she-may-rein-edd9f99be63d

Stacking

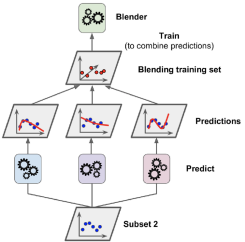
- Instead of using trivial functions to aggregate the predictions of all predictors in an ensemble, why don't we train a model to perform this aggregation?



Aggregating predictions using a blending predictor



Training the first layer



Training the blender

Comparison of Ensemble Methods

Method	Strengths	Limitations
Voting	Combines different algorithms so models can complement each other; simple to implement and quick to try.	If base models differ greatly in quality, weak ones can drag down overall performance.
Bagging	Lowers variance of high-variance learners, helping prevent overfitting; supports parallel training efficiently.	Limited effect for high-bias learners; all models share the same inductive bias (same base algorithm).
Boosting	Sequentially focuses on errors to correct them; can capture complex patterns and achieve high accuracy.	Sensitive to noise/outliers with risk of overfitting; training time and tuning complexity can be high.
Stacking	Can achieve strong performance by leveraging complementary models; meta-learner can improve generalization.	Requires careful meta-model choice and data-splitting strategy; structure is complex and harder to implement.

Thank You!

(Q & A)

References

- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). An Introduction to Statistical Learning: with Applications in R. New York: Springer.
- Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media, Inc.
- Advanced Methods for Data Analysis [Lecture Notes]. Retrieved from http://www.stat.cmu.edu/ ryantibs/advmethods/notes/highdim.pdf